

ml-task-03

June 20, 2025

```
[3]: import pandas as pd
      from sklearn.model_selection import train_test_split, GridSearchCV
      from sklearn.linear_model import LogisticRegression
      from sklearn.preprocessing import StandardScaler, OneHotEncoder
      from sklearn.compose import ColumnTransformer
      from sklearn.pipeline import Pipeline
      from sklearn.impute import SimpleImputer
      from sklearn.metrics import classification_report, confusion_matrix
      from imblearn.pipeline import Pipeline as ImbPipeline
      from imblearn.over_sampling import SMOTE
```

```
[4]: # --- Load the data ---
      application_df = pd.read_csv("application_record.csv")
      credit_df = pd.read_csv("credit_record.csv")
```

```
[5]: # --- Generate binary target labels ---
      credit_df["label"] = credit_df["STATUS"].apply(lambda x: 1 if x in ["3", "4", "5"] else 0)
      user_labels = credit_df.groupby("ID")["label"].max().reset_index()

      # --- Merge with application data ---
      data = pd.merge(application_df, user_labels, on="ID", how="inner")
```

```
[6]: # --- Split features and target ---
      X = data.drop(columns=["ID", "label"])
      y = data["label"]
```

```
[7]: # --- Identify feature types ---
      categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()
      numerical_cols = X.select_dtypes(exclude=["object"]).columns.tolist()
```

```
[8]: # --- Preprocessing pipelines ---
      numeric_transformer = Pipeline(steps=[
          ("imputer", SimpleImputer(strategy="median")),
          ("scaler", StandardScaler())
      ])
```

```
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numerical_cols),
    ("cat", categorical_transformer, categorical_cols)
])
```

```
[9]: # --- Build pipeline with SMOTE and Logistic Regression ---
pipeline = ImbPipeline(steps=[
    ("preprocessor", preprocessor),
    ("smote", SMOTE(random_state=42)),
    ("classifier", LogisticRegression(solver="liblinear"))
])
```

```
[10]: # --- Hyperparameter tuning ---
param_grid = {
    "classifier__C": [0.01, 0.1, 1, 10],
    "classifier__penalty": ["l1", "l2"]
}
```

```
[11]: # --- Train/test split ---
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,
    ↪ test_size=0.2, random_state=42)
```

```
[12]: # --- Run GridSearchCV ---
grid_search = GridSearchCV(pipeline, param_grid, cv=3, scoring="f1", n_jobs=-1)
grid_search.fit(X_train, y_train)
```

```
[12]: GridSearchCV(cv=3,
                  estimator=Pipeline(steps=[('preprocessor',
                                             ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
                  SimpleImputer(strategy='median')),
                  ('scaler',
                     StandardScaler()))]),
                  ['CNT_CHILDREN',
                  'AMT_INCOME_TOTAL',
                  'DAYS_BIRTH',
                  'DAYS_EMPLOYED',
                  'FLAG_MOBIL',
                  'FLAG_WORK_PHONE',
                  'FLAG_PHONE',
                  'FLAG_EMAIL',
                  'CNT_FAM_MEMBERS'])),
```

```

('cat',
Pipeline(steps=[
    OneHotEncoder(handle_unknown='ignore'))],
[('CODE_GENDER',
'FLAG_OWN_CAR',
'FLAG_OWN_REALTY',
'NAME_INCOME_TYPE',
'NAME_EDUCATION_TYPE',
'NAME_FAMILY_STATUS',
'NAME_HOUSING_TYPE',
'OCCUPATION_TYPE')]),
('smote', SMOTE(random_state=42)),
('classifier',
LogisticRegression(solver='liblinear'))],
n_jobs=-1,
param_grid={('classifier__C': [0.01, 0.1, 1, 10],
'classifier__penalty': ['l1', 'l2'])},
scoring='f1')

```

```

[13]: # --- Evaluate best model ---
y_pred = grid_search.predict(X_test)

print("Best Parameters:", grid_search.best_params_)
print("\nClassification Report:\n", classification_report(y_test, y_pred,
↳ digits=4))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

Best Parameters: {'classifier__C': 0.1, 'classifier__penalty': 'l1'}

Classification Report:

	precision	recall	f1-score	support
0	0.9948	0.6332	0.7738	7232
1	0.0134	0.6000	0.0262	60
accuracy			0.6329	7292
macro avg	0.5041	0.6166	0.4000	7292
weighted avg	0.9867	0.6329	0.7677	7292

Confusion Matrix:

```

[[4579 2653]
 [ 24 36]]

```